

Analiza Arhitecturală și Tehnică

“ExchangePoint”

A) Introducere și Contextul Proiectului

Proiectul propune o soluție software pentru administrarea unei case de schimb valutar, o problemă care, deși pare simplă la nivel conceptual, implică o gestionare riguroasă a fluxurilor de date și a regulilor de business. Obiectivul aplicației este de a simula interacțiunile dintre trei categorii de utilizatori (**Owner**, **Cashier**, **Client**) și de a menține o evidență clară a lichidităților și a profitului. Din analiza codului sursă, reiese o arhitectură modulară, bazată pe principii solide de programare orientată pe obiecte (**POO**), menită să asigure atât extensibilitatea sistemului, cât și siguranța operațiunilor financiare.

B) Organizarea Codului și Designul Modular

Primul aspect care merită menționat este organizarea proiectului. Separarea strictă între fișierele header (**.h**) și fișierele de implementare (**.cpp**) indică o abordare profesionistă. În fișiere precum **ExchangeShop.cpp** sau **User.cpp**, se observă o structură clară, unde responsabilitățile sunt distribuite echilibrat.

Clasa centrală, **ExchangeShop**, acționează ca un container și manager pentru toate entitățile sistemului. Aceasta încapsulează logica de calcul, istoricul cursurilor (**CurrencyHistory**) și colecția de tranzacții. Utilizarea structurilor auxiliare în **extra.h**, precum **Date** și **ExchangeRate**, simplifică logica internă a claselor mari și oferă o abstractizare utilă pentru manipularea timpului și a valorilor monetare.

C) Ierarhia Utilizatorilor și Gestiunea Drepturilor prin NVI

Ierarhia clasei **User** este una dintre cele mai interesante părți ale proiectului. Autorul a ales să definească **User** ca o clasă abstractă, ceea ce este logic, deoarece nu poți avea un „utilizator general” fără un rol specific.

Un detaliu tehnic de finețe este utilizarea idiomului **Non-Virtual Interface(NVI)** în `User.h`. Metodele publice `getTransactionHistory` și `printTransactionHistory` sunt ne-virtuale, dar ele apelează intern metode virtuale protejate precum `filterTransactionHistory`. Această abordare permite clasei de bază să garanteze un anumit comportament predefinit (de exemplu, încărcarea datelor din magazin), în timp ce derivatele (`Owner`, `Cashier`) personalizează doar modul în care acele date sunt filtrate sau afișate.

În `User.cpp`, observăm și interzicerea copierii obiectelor de tip utilizator prin ștergerea constructorului de copiere (= `delete`). Aceasta este o decizie de design matură, prevenind duplicarea logică a unor entități care ar trebui să fie unice în sistem, cum este cazul `Owner`-ului.

D) Ierarhia Tranzacțiilor și Clonarea Polimorfică

Sistemul de tranzacții (`Exchange`, `Deposit`, `Withdraw`) reprezintă un exemplu clasic de polimorfism. Fiecare tranzacție moștenește clasa de bază `Transaction`, dar implementează în mod propriu calculul profitului și afișarea.

Un punct critic rezolvat corect este clonarea polimorfică. Deoarece tranzacțiile sunt stocate în `ExchangeShop` folosind `std::vector<std::shared_ptr<Transaction>>`, copierea întregului magazin (necesară pentru operatorul de atribuire) ar fi fost imposibilă fără metoda virtuală `clone()`. În `Exchange.cpp` sau `Withdraw.cpp`, vedem implementări de tipul `return std::make_shared<Withdraw>(*this);`. Această tehnică permite duplicarea obiectelor prin pointeri la clasa de bază, fără a pierde tipul concret al obiectului, evitând astfel problema „slicing”-ului (tăierea obiectului).

E) Managementul Resurselor: Smart Pointers și Copy-and-Swap

Trecerea către un stil de programare modern este evidentă prin utilizarea `std::shared_ptr`. În loc de pointeri bruti care ar fi necesitat destructori complecși și ar fi crescut riscul de scurgeri de memorie, autorul folosește smart pointers care gestionează automat durata de viață a obiectelor.

În `ExchangeShop.cpp`, implementarea operatorului de atribuire folosește idiomul *copy-and-swap*. Aceasta presupune crearea unei copii temporare a

obiectului și interschimbarea resurselor folosind o funcție swap prietenă. Este cea mai sigură metodă de a implementa atribuirea în C++, deoarece oferă garanții puternice în caz de excepții: dacă alocarea memoriei pentru copie eșuează, starea obiectului original rămâne intactă.

F) Tratarea Erorilor și Ierarhia de Excepții

Aplicația nu se bazează pe returnarea de coduri de eroare, ci pe un sistem robust de excepții, definit în **Exception.h**. Faptul că toate excepțiile derivă din **ExchangeException** (care la rândul ei derivă din **std::runtime_error**) permite o capturare ierarhică a erorilor.

În **main.cpp**, blocurile try-catch protejează execuția programului. De exemplu, dacă un utilizator introduce un format de dată greșit, **Date::Date(const std::string& iso_ymd)** din **extra.cpp** aruncă o **ValidationException**. Programul prinde această eroare, afișează un mesaj sugestiv și revine în meniu, în loc să se închidă forțat. Aceasta este o dovadă de atenție la detalii și de orientare către utilizator.

G) Logica de Business: Calculul Profitului și Validări

Implementarea calculului de profit în **Exchange.cpp** este remarcabilă prin detaliere. Metoda **profitCalculation** ține cont de valuta principală a magazinului și extrage cursurile de cumpărare și vânzare din istoric la data tranzacției. Există validări care previn calculul dacă datele sunt incomplete (cursuri de 0.0), ceea ce previne obținerea unor rezultate financiare eronate.

De asemenea, în **ExchangeShop.cpp**, metodele **withdrawMoney** și **exchangeMoney** conțin validări critice de lichiditate. Dacă magazinul nu are destui bani într-o anumită valută, se aruncă o **TransactionConstraintException**. Acest mecanism asigură că magazinul nu poate ajunge niciodată pe „minus”, respectând regulile de funcționare ale unei entități financiare reale.

H) Persistența Datelor și Modul Replay

O funcționalitate care adaugă multă valoare proiectului este capacitatea de a salva și reîncărca tranzacțiile dintr-un fișier text (**shop_data.txt**). În **main.cpp**, funcția **replayOperations** simulează o bază de date simplă. La fiecare pornire, programul poate „re-executa” tranzacțiile salvate pentru a reconstrui starea lichidităților și a istoricului. Acest mod de lucru, combinat cu posibilitatea de a rula în regim interactiv sau demonstrativ, face ca aplicația să fie ușor de testat și verificat.

I) Stilul de Cod și Lizibilitatea

Codul este scris într-un stil curat, cu nume de variabile sugestive (ex: **sum_in**, **currency_out_name**, **clamped_tax**). Comentariile sunt prezente acolo unde logica este mai complexă, cum ar fi explicarea motivului pentru care se folosește un anumit tip de cast (**dynamic_pointer_cast** în **printTransactionStats**). Deși downcast-ul este uneori privit cu scepticism în POO, în acest context este folosit „cu sens”, pentru a separa raportarea statistică a tipurilor de tranzacții.

Un alt aspect pozitiv este utilizarea **const** peste tot unde starea obiectului nu trebuie modificată. Această practică („const correctness”) ajută compilatorul să optimizeze codul și ajută programatorul să evite bug-uri logice prin care ar putea modifica din greșeală datele magazinului într-o funcție de afișare.

J) Observații Critice și Posibile Îmbunătățiri

În ciuda complexității sale, există zone unde proiectul ar putea evolua. De exemplu, managementul datei în **extra.cpp** este făcut manual. Deși este o implementare corectă a algoritmului pentru ani bisecți și număr de zile pe lună, într-un mediu de producție s-ar putea prefera librăria **<chrono>**. De asemenea, stocarea în fișier text este vulnerabilă la editări manuale ale utilizatorului; un format mai structurat (ca **JSON**) ar fi oferit o mai bună integritate a datelor.

K) Concluzie

În concluzie, proiectul „**ExchangePoint**” este o lucrare solidă, care demonstrează o stăpânire avansată a limbajului C++ și a paradigmei de programare orientată pe obiecte. Nu este doar un exercițiu algoritmic, ci o încercare reușită de a modela o problemă reală folosind abstractizări corecte. Utilizarea smart pointerilor, a ierarhiilor polimorifice cu clonare și a sistemului de excepții ridică proiectul mult peste nivelul de bază, reflectând capacitatea studentului de a scrie cod robust, mentenabil și sigur. Este un exemplu de bună practică în design-ul software, unde fiecare clasă are o responsabilitate bine definită, iar interacțiunile dintre ele sunt gestionate prin interfețe clare.